

This end-to-end guide covers setting up **SimBricks** with **QEMU** and the **Verilated Corundum NIC** (component-corundum) for custom RTL development on either **Windows (WSL2)** or **macOS**. It includes all required fixes for upstream packaging changes, WSL2 environment quirks, initramfs booting, and timing synchronization.

1. Platform Prerequisites (Windows WSL2 vs. macOS)

SimBricks' pre-built Conda packages

(<https://conda.simbricks.io/latest>) and guest kernel images target x86_64 **Linux** (linux-64).

Option A: Windows 10/11 with WSL2 (Recommended)

1. Enable nested virtualization in C:\Users\<YourUsername>\.wslconfig:

Ini, TOML

[wsl2]

nestedVirtualization=true

2. Restart WSL (wsl --shutdown in PowerShell), open your Ubuntu WSL2 terminal, and grant your user access to /dev/kvm:

Bash

```
sudo usermod -aG kvm $USER
```

```
sudo chmod 666 /dev/kvm
```

Option B: macOS (Intel or Apple Silicon)

Because SimBricks binaries and libguestfs require an x86_64 Linux environment, run the workflow inside an Ubuntu 22.04 x86_64 Linux VM using **Lima** (or UTM/Multipass):

1. Install Lima via Homebrew and launch an x86_64 Ubuntu VM:

Bash

```
brew install lima
```

```
limactl start --name=simbricks --arch=x86_64 --cpus=6 --memory=12 --disk=50
```

```
template://ubuntu-22.04
```

```
limactl shell simbricks
```

2. **Note for Apple Silicon (M-series) Macs:** Because an ARM64 host cannot provide hardware `/dev/kvm` for an x86_64 guest kernel, disable KVM acceleration on QEMU inside your Python simulation script (`host_sim.accel = []`) or run in synchronized mode via `sim.enable_synchronization()` and pass `QEMU_ACCEL=tcg` when building the disk image in Step 4.

2. Install Host Dependencies & Create the Conda Environment

Inside your Linux terminal (WSL2 or Lima VM), install the C++/Verilator build toolchain, Packer, libguestfs-tools, and Micromamba:

Bash

```
sudo apt-get update
sudo apt-get install -y build-essential git ca-certificates curl bzip2 \
    verilator libboost-all-dev python3 python3-pip python3-venv \
    qemu-system-x86 qemu-utils libguestfs-tools linux-image-generic
```

```
# Ensure a readable kernel exists in /boot so libguestfs (virt-copy-out) succeeds on WSL2
sudo chmod +r /boot/vmlinuz*
```

```
# Install HashiCorp Packer (used to build the guest disk image with mqn timer)
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt-get update && sudo apt-get install -y packer
```

Install Micromamba

```
"${SHELL}" <(curl -L micro.mamba.pm/install.sh)
source ~/.bashrc
```

Create and activate the simbricks environment with the pre-built SimBricks libraries and simulators:

Bash

```
micromamba create -n simbricks python=3.11 \
```

```
-c https://conda.simbricks.io/latest \
-c conda-forge \
simbricks-lib \
simbricks-orchestration \
simbricks-local \
simbricks-qemu-sim-bin \
simbricks-qemu-sim-py \
simbricks-net-base-sim-bin \
simbricks-net-base-sim-py
```

micromamba activate simbricks

3. Build the Verilated Corundum NIC From Source

Clone simbricks/component-corundum so you can modify the Verilog/SystemVerilog RTL and C++ PCIe/Ethernet adapter locally:

Bash

```
mkdir -p ~/simbricks-ws && cd ~/simbricks-ws
git clone https://github.com/simbricks/component-corundum.git
cd component-corundum
git submodule update --init --recursive
```

1. Verilate the Corundum RTL and compile the SimBricks C++ adapter

```
make corundum-build \
  SIMBRICKS_INC_DIR="$CONDA_PREFIX/include" \
  SIMBRICKS_LIB_DIR="$CONDA_PREFIX/lib"
```

2. Install the simb_corundum binary into your active Conda PATH

```
make corundum-install \
  PREFIX="$CONDA_PREFIX" \
  SIMBRICKS_INC_DIR="$CONDA_PREFIX/include" \
  SIMBRICKS_LIB_DIR="$CONDA_PREFIX/lib"
```

3. Install the Python orchestration bindings in editable mode

```
make corundum-python-develop PYTHON=python
```

4. Build the Guest Disk Image with the mqnic Driver

Corundum requires its out-of-tree mqnic.ko Linux kernel module inside the guest image.

Use simbricks/image-builder with two critical fixes:

1. Patch examples/corundum/install-mqnic.sh so it installs git and ca-certificates inside the provisioning VM.
2. Explicitly pass NAME=base to make image (WSL2 exports NAME=<YourHostname> by default, which otherwise renames the output directory).

Bash

```
cd ~/simbricks-ws
```

```
git clone https://github.com/simbricks/image-builder.git
```

```
cd image-builder
```

```
# Fix missing git dependency in the upstream provisioning script
```

```
sed -i 's/build-essential/build-essential git ca-certificates/' examples/corundum/install-mqnic.sh
```

```
# Build the image (takes ~6-7 minutes with KVM)
```

```
# Note for Apple Silicon Macs: append QEMU_ACCEL=tcg to this command
```

```
make image NAME=base EXTRA_SCRIPTS="examples/corundum/install-mqnic.sh"
```

```
# Copy the built artifacts into the global_input directory structure
```

```
mkdir -p ~/simbricks-ws/global_input/images/base
```

```
cp -r ~/simbricks-ws/image-builder/output/base/* ~/simbricks-ws/global_input/images/base/
```

```
ln -sf ~/simbricks-ws/global_input/images/base/base ~/simbricks-ws/global_input/images/base/base.qcow2
```

```
Verify that ~/simbricks-ws/global_input/images/base/ contains base, base.qcow2, and boot/ (vmlinuz and initrd*).
```

5. Create the Virtual Prototype Script (corundum_qemu_vp.py)

Create ~/simbricks-ws/corundum_qemu_vp.py. This script includes three important adaptations for the current SimBricks runtime:

- Uses system.ExternalDiskImage to load your local mqnic-enabled disk image instead of fetching a stock image over HTTP.

- Patches `qemu_sim.QemuSim.run_cmd` **in-place** (avoiding subclassing, which breaks `simple_simulation`'s `==` class check) to pass `-initrd` so the guest kernel loads its ahci SATA driver and mounts `/dev/sda1`.
- Optionally exposes a shared folder (`~/simbricks-ws/guest_share -> /mnt/host`) and a toggle for **lockstep timing synchronization**.

Bash

```
mkdir -p ~/simbricks-ws/guest_share
```

```
cat << 'EOF' > ~/simbricks-ws/corundum_qemu_vp.py
```

```
import os
```

```
from pathlib import Path
```

```
from simbricks.orchestration import system
```

```
from simbricks.orchestration.helpers import simulation as sim_helpers
```

```
from simbricks.orchestration.helpers import instantiation as inst_helpers
```

```
from simbricks.components.qemu import simulation as qemu_sim
```

```
from simbricks.components.net.simulation import base as net_sim
```

```
from simbricks.components.corundum import system as corundum_sys
```

```
from simbricks.components.corundum import simulation as corundum_sim
```

```
# Toggle between fast functional KVM mode (False) and cycle-synchronized TCG mode (True)
```

```
SYNCHRONIZED_TIMING = False
```

```
# 1. System Configuration (Hardware & Topology)
```

```
syst = system.System("Corundum-QEMU-Prototype")
```

```
img_dir = Path.home() / "simbricks-ws" / "global_input" / "images" / "base"
```

```
share_dir = Path.home() / "simbricks-ws" / "guest_share"
```

```
base_disk = system.ExternalDiskImage(
```

```
    syst,
```

```
    path=str(img_dir / "base"),
```

```
    boot_dir=str(img_dir / "boot"),
```

```
)
```

```
# Patch QemuSim.run_cmd in-place to attach initrd, 9p shared folder, and telnet serial ports
```

```
_orig_run_cmd = qemu_sim.QemuSim.run_cmd
```

```
_port_counter = 4444
```

```
def _patched_run_cmd(self, inst):
```

```
    global _port_counter
```

```
    cmd = _orig_run_cmd(self, inst)
```

```
    initrd = next(p for p in sorted((img_dir / "boot").glob("initrd*")) if p.exists())
```

```
    port = _port_counter
```

```
    _port_counter += 1
```

```
    extra = (
```

```
        f" -initrd {initrd}"
```

```
        f" -virtfs
```

```
local,path={share_dir},mount_tag=hostshare,security_model=none,id=hostshare"
```

```
        f" -serial telnet:127.0.0.1:{port},server,nowait"
```

```
    )
```

```
    return f"{cmd}{extra}" if isinstance(cmd, str) else [*cmd, *extra.split()]
```

```
qemu_sim.QemuSim.run_cmd = _patched_run_cmd
```

```
# Switch connecting the two NICs
```

```
switch = system.EthSwitch(syst)
```

```
# Host 0 (Server) + Corundum NIC 0
```

```
host0 = corundum_sys.CorundumLinuxHost(syst)
```

```
host0.add_disk(base_disk)
```

```
host0.add_disk(system.LinuxConfigDiskImage(syst, host0))
```

```
nic0 = corundum_sys.CorundumNIC(syst)
```

```
nic0.add_ipv4("10.0.0.1")
```

```
host0.connect_pcie_dev(nic0)
```

```
switch.connect_eth_peer_if(nic0._eth_if)
```

```
# Host 1 (Client) + Corundum NIC 1
```

```
host1 = corundum_sys.CorundumLinuxHost(syst)
```

```
host1.add_disk(base_disk)
```

```
host1.add_disk(system.LinuxConfigDiskImage(syst, host1))
```

```
nic1 = corundum_sys.CorundumNIC(syst)
```

```

nic1.add_ipv4("10.0.0.2")
host1.connect_pcie_dev(nic1)
switch.connect_eth_peer_if(nic1._eth_if)

# Default Automated Workload: Iperf TCP test across both Verilated Corundum NICs
server_app = system.IperfTCPServer(h=host0)
host0.add_app(server_app)

client_app = system.IperfTCPClient(h=host1, server_ip=nic0._ip)
client_app.wait = True
host1.add_app(client_app)

# 2. Simulation Configuration (Map Components to Simulators)
sim = sim_helpers.simple_simulation(
    syst,
    compmap={
        system.FullSystemHost: qemu_sim.QemuSim,
        corundum_sys.CorundumNIC: corundum_sim.CorundumVerilatorNICSim,
        system.EthSwitch: net_sim.SwitchNet,
    },
)

if SYNCHRONIZED_TIMING:
    # Locks QEMU (-icount TCG), Corundum RTL, and SwitchNet to a shared 500ns barrier
    sim.enable_synchronization()

# 3. Instantiation Configuration
instance = inst_helpers.simple_instantiation(sim)
instantiations = [instance]
EOF

```

6. Running the Simulation & Inspecting Results

Run the Simulation

Bash

```
cd ~/simbricks-ws
```

```
simbricks-run --verbose --global-input-dir ~/simbricks-ws/global_input
```

```
corundum_qemu_vp.py
```

Print Guest Console & iperf Output from the JSON Log

SimBricks collects all simulator outputs into an out.json file. Extract the guest console logs with:

Bash

```
python -c "  
import json, glob  
out_file = sorted(glob.glob('**/out*.json', recursive=True) + glob.glob('*.json'))[-1]  
data = json.load(open(out_file))  
for sim_name, logs in data.get('sims', data).items():  
    if isinstance(logs, dict) and 'stdout' in logs:  
        print(f'\n=== {sim_name} ===\n' + '\n'.join(logs['stdout'][-30:]))  
"
```

Interactive Shell & Custom Packet Generation (Optional)

To open an interactive terminal inside host1 and run scripts directly from ~/simbricks-ws/guest_share, replace the lperfTCPCClient block in corundum_qemu_vp.py with:

Python

```
client_app = system.GenericRawCommandApplication(  
    h=host1,  
    cmds=[  
        "mkdir -p /mnt/host && mount -t 9p -o trans=virtio hostshare /mnt/host",  
        "setsid /bin/bash </dev/ttyS1 >/dev/ttyS1 2>&1",  
    ],  
)  
client_app.wait = True  
host1.add_app(client_app)
```

Then run telnet 127.0.0.1 4445 in a second terminal window once the guest boots.

7. Iterative RTL Modification Workflow

Mode	Configuration	Boot Time	Use Case
Unsynchronized (Fast Functional)	SYNCHRONIZED_TIMING = False (uses KVM)	~4 seconds	Functional verification of new

Mode	Configuration	Boot Time	Use Case
			RTL modules, custom headers, MMIO registers, and driver changes
Synchronized (Lockstep Timing)	SYNCHRONIZED_TIMING = True (uses QEMU - icount)	~1.5–3 minutes	Cycle-accurate PCIe/Ethernet co-simulation, latency measurements, and throughput benchmarking

Whenever you modify Verilog/SystemVerilog files under `~/simbricks-ws/component-corundum/corundum/fpga/` or the C++ testbench wrapper at `~/simbricks-ws/component-corundum/adaptor/corundum_simbricks_adapter.cpp`, recompile and reinstall `simb_corundum`:

Bash

```
cd ~/simbricks-ws/component-corundum
make corundum-install \
  PREFIX="$CONDA_PREFIX" \
  SIMBRICKS_INC_DIR="$CONDA_PREFIX/include" \
  SIMBRICKS_LIB_DIR="$CONDA_PREFIX/lib"
```

Then re-run `simbricks-run` in `~/simbricks-ws`—no disk image rebuild is required unless you modify the C kernel driver (`corundum/modules/mqnic/`).